

GEORGETOWN · DSAN 6725 · SPRING 2026

Self-Constructing Agents

A domain-agnostic framework for building, verifying, and accumulating reusable skills and tools in code-acting language-model agents.

Ihsan Alaeddin · Sazan Khalid

DEPARTMENT OF DATA SCIENCE AND ANALYTICS

The Problem

Why agents need to *remember* and *improve*.



PROBLEM 1

Cold start every task

An agent that solves a task today loses that work tomorrow. The conversation log is the only artifact, and no future agent reads it.



PROBLEM 2

No domain compounding

Specialised knowledge — file formats, validation rules, repository conventions — has to be re-derived on every call. The agent never becomes an expert.



PROBLEM 3

Unverified output

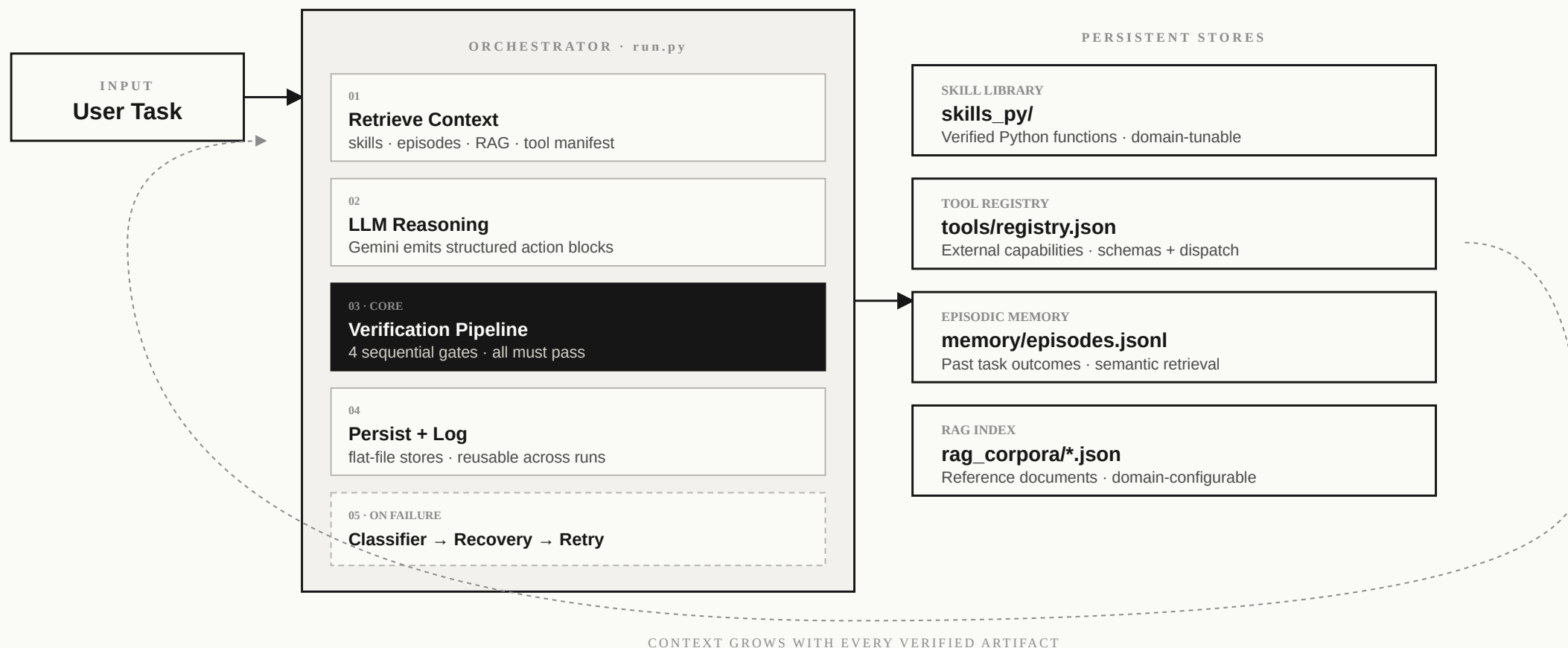
When the agent invents an API call that does not exist, nothing catches it. Hallucinations are silently accepted as correct code.

OUR APPROACH

***A domain-agnostic* framework where the agent writes verified Python skills and tools as a by-product of solving tasks — and reuses them across future sessions, in any domain.**

End-to-End Flow

An orchestrator that *builds and verifies* on every task.



Skills vs Tools

Pure computation versus *boundary-crossing*.

Skill

PURE · INTERNAL

WHAT IT IS

A pure Python function that operates on data already in memory. Imported and called **directly** by the agent's generated code.

WHEN TO USE

Computation, transformation, validation. Anything that produces output from input without external side effects.

EXAMPLES

```
validate_iban(iban)
```

```
parse_pain001_fields(xml)
```

```
compute_moving_average(prices)
```

STORAGE

skills_py/ — one file per skill, metadata header + function + unit test.

Tool

EXTERNAL · MEDIATED

WHAT IT IS

A function that crosses a boundary — network, filesystem, external API. Invoked through **structured calls** mediated by the orchestrator.

WHEN TO USE

Live data fetching, file I/O, third-party API calls. Anything that reaches outside the agent's Python process.

EXAMPLES

```
fetch_stock_price(ticker)
```

```
read_file(path)
```

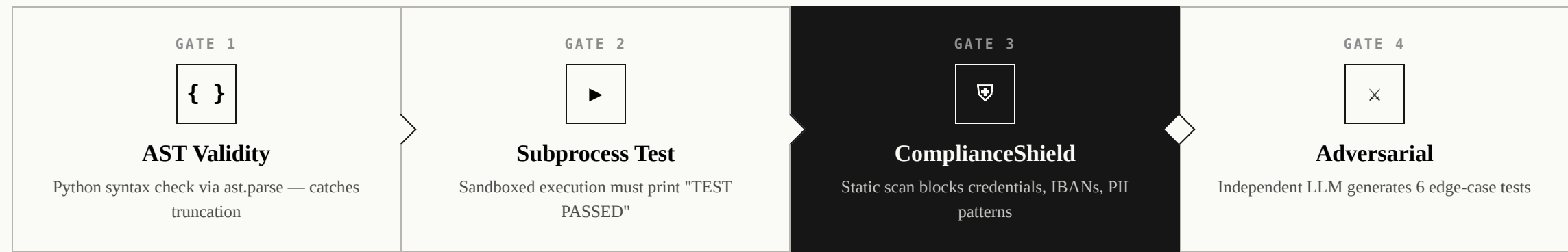
```
fetch_weather(city)
```

STORAGE

tools/registry.json — name, input schema, output schema, source path, usage stats.

The Four-Gate Pipeline

Every artifact must pass *four sequential gates*.



Failure at any gate → artifact discarded → retry with feedback (up to 3 attempts)

IF ALL FOUR PASS

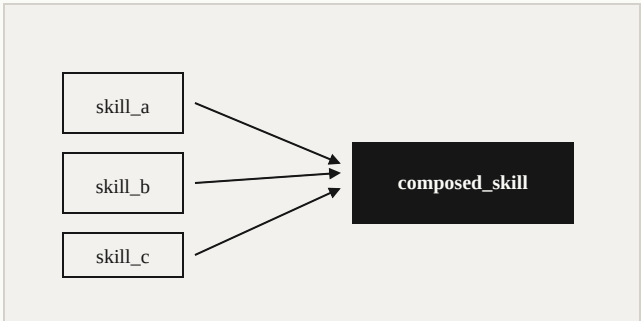
written to disk · registered in index · available for all future tasks

Skill Lifecycle Mechanics

The library *compounds* — and stays lean.

MECHANIC 1

Composition



Before generating new code, the orchestrator asks: can this be solved by combining existing skills? If yes, the composition itself becomes a verified skill.

TRIGGER: every new task, before generation

MECHANIC 2

Evolution

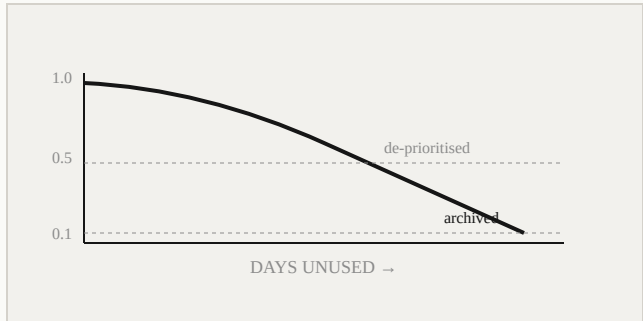


When a skill's failure count exceeds its success count, the agent prompts itself to diagnose and patch the broken function. The patched version re-runs through all four gates.

TRIGGER: fail_count ≥ 2 AND fail > success

MECHANIC 3

Decay



Each skill carries a relevance score that decays with disuse. Below 0.5 over 7 days the skill is de-prioritised; below 0.1 over 14 days it is auto-archived.

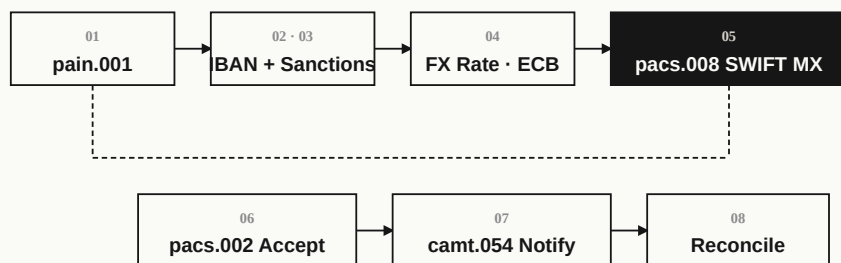
SCORE: success / (1 + days_unused × 0.1)

Two Domains, One Framework

From a tuned domain to *any* user repository.

■ DEMO 1 · IMPLEMENTED

ISO 20022 Cross-Border Payment

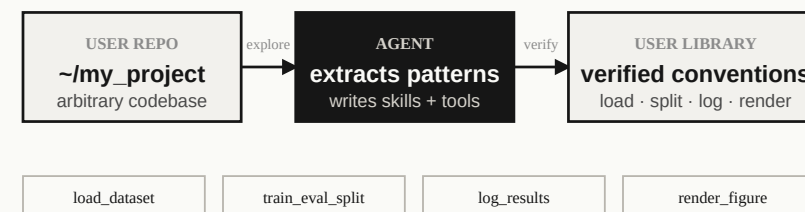
Meridian Ltd (UK) → Fischer GmbH (DE) · €47,832.50

END-TO-END ID PRESERVED ACROSS ALL 8 STEPS

A real interbank payment workflow executed against the agent's domain-tuned skill library.
10 verified skills covering ISO 20022, NACHA, and FX conversion.

□ DEMO 2 · LIVE · DOMAIN-AGNOSTIC

Point at Any Repository

User-supplied codebase · convention learning · live tool authoring

load_dataset

train_eval_split

log_results

render_figure

YOUR CONVENTIONS · NOW VERIFIED · NOW REUSABLE

The same architecture, applied to a domain it has never seen. **Institutional knowledge made portable** — your conventions become a verified, reusable library.

Agents that *remember* and *improve*.

4

VERIFICATION GATES

3

LIFECYCLE MECHANICS

∞

DOMAINS SUPPORTED

*Procedural knowledge in code-acting agents should be **persistent, verified, and lifecycle-managed** — not ephemeral and conversational. We built the framework that does this, and demonstrated it on payment-domain integration and on arbitrary user repositories.*